

ADR-02: Estrategia de persistencia de datos

ZenFlow — JSON ahora, SQLite cuando sea necesario

Campo	Valor
Autor	Dylan Vazquez
Fecha	05/06/2025
Estado	Propuesto

Contexto

ZenFlow necesita guardar dos tipos de datos: tareas y hábitos. Desde el inicio del proyecto sabía que esos datos tenían que vivir en algún lado — en memoria no funciona porque se pierden al cerrar la app. La pregunta era: ¿dónde y cómo?

Las condiciones que influyeron en esta decisión fueron concretas:

- El proyecto está en etapa temprana. Las estructuras de datos todavía pueden cambiar — una Tarea puede necesitar campos nuevos, un Hábito puede necesitar más propiedades. Comprometerse con una base de datos relacional desde el inicio significa tener que hacer migraciones de esquema cada vez que cambia algo.
- Trabajo solo y el tiempo del semestre es limitado. Configurar un motor de base de datos al principio me quitaría tiempo que necesito para la arquitectura y la lógica de negocio.
- ZenFlow es una app local de escritorio. No hay múltiples usuarios, no hay concurrencia, no hay necesidad de acceso remoto. La complejidad de una base de datos cliente-servidor no está justificada para este caso de uso.
- Ya apliqué el patrón repositorio con interfaces (ITareaRepo, IHábitoRepo), lo que significa que puedo cambiar la implementación de almacenamiento en el futuro sin tocar nada de la lógica de negocio.

Decisión

Usar archivos JSON como mecanismo de persistencia en la versión inicial de ZenFlow, con la arquitectura preparada para migrar a SQLite cuando el proyecto lo requiera.

En la práctica esto significa:

- TareaRepoJson guarda y lee tareas desde tareas.json
- HábitoRepoJson guarda y lee hábitos desde habitos.json
- Ambas implementan sus respectivas interfaces (ITareaRepo e IHábitoRepo)
- Cuando se migre a SQLite, se crean TareaRepoSQLite y HábitoRepoSQLite que implementan las mismas interfaces — sin tocar GestorTareas ni GestorHábitos

¿Por qué?

La razón principal no es que JSON sea mejor que SQLite — es que JSON es suficiente para donde está el proyecto ahora. Empezar con la solución más simple que funcione y migrar cuando sea necesario es una decisión arquitectónica consciente, no una improvisación.

JSON permite iterar rápido: puedo ver el contenido del archivo directamente, no necesito un cliente de base de datos, y si el esquema cambia simplemente actualizo la clase. Cuando el volumen de datos crezca o necesite consultas más complejas, la interfaz ITareaRepo actúa como firewall — cambio la implementación sin que nadie más se entere.

Alternativas consideradas

Alternativa	Por qué la descarté
MySQL / PostgreSQL	Requieren un servidor corriendo por separado. ZenFlow es una app de escritorio local — instalar un servidor de base de datos para guardar tareas personales es excesivo y complica innecesariamente el setup para el usuario.
MongoDB	Es NoSQL orientada a documentos. Aunque parece similar a JSON, agrega complejidad de configuración sin ninguna ventaja real para datos tan simples como tareas y hábitos. Tampoco conozco bien su ecosistema en .NET.
SQL Server	Es el motor de Microsoft y encaja bien con .NET, pero también requiere instalación de servidor y licencias. Para una app personal de escritorio es demasiado pesado.
Empezar directo con SQLite	Era una opción válida, pero SQLite requiere más configuración inicial que JSON. Como el proyecto está en etapa temprana y las estructuras de datos pueden cambiar, empezar con JSON me permite iterar rápido sin preocuparme por migraciones de esquema.

Consecuencias

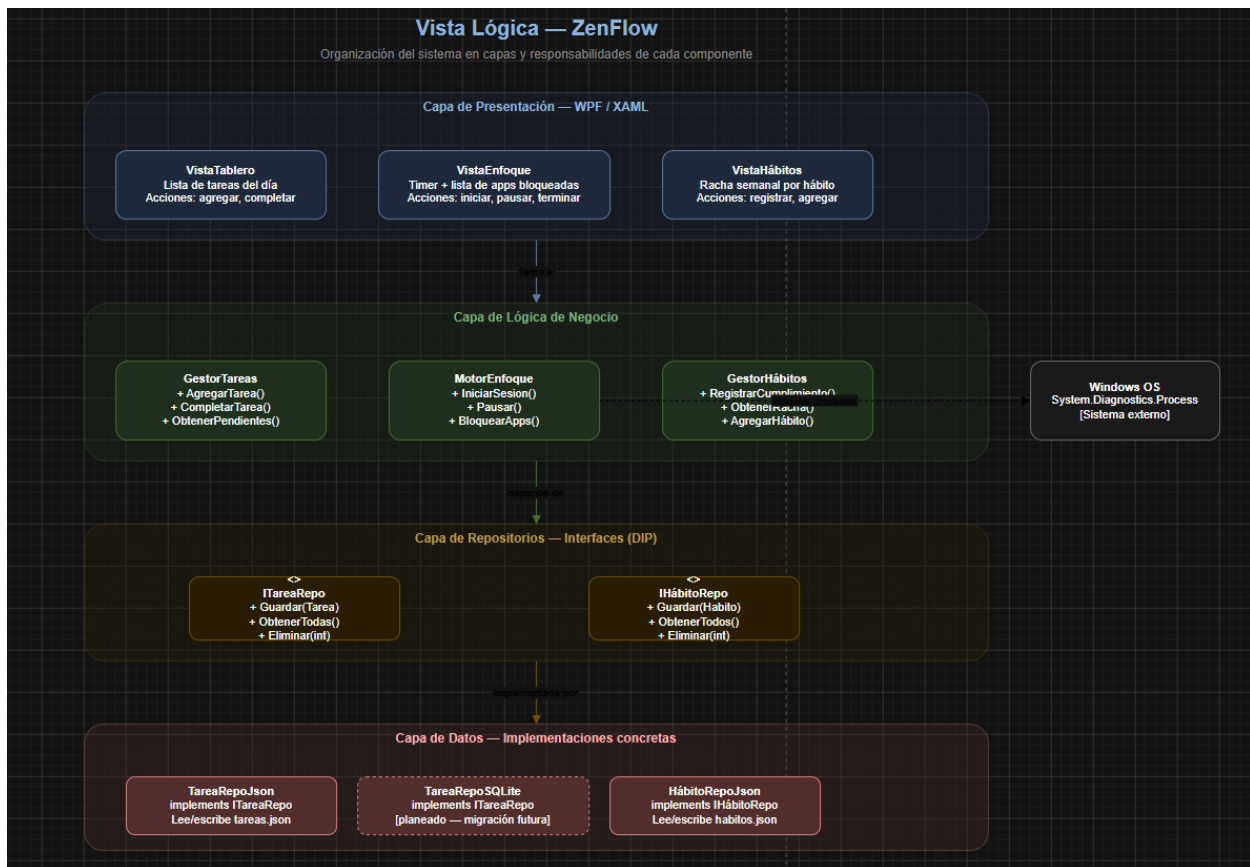
Lo que gano:

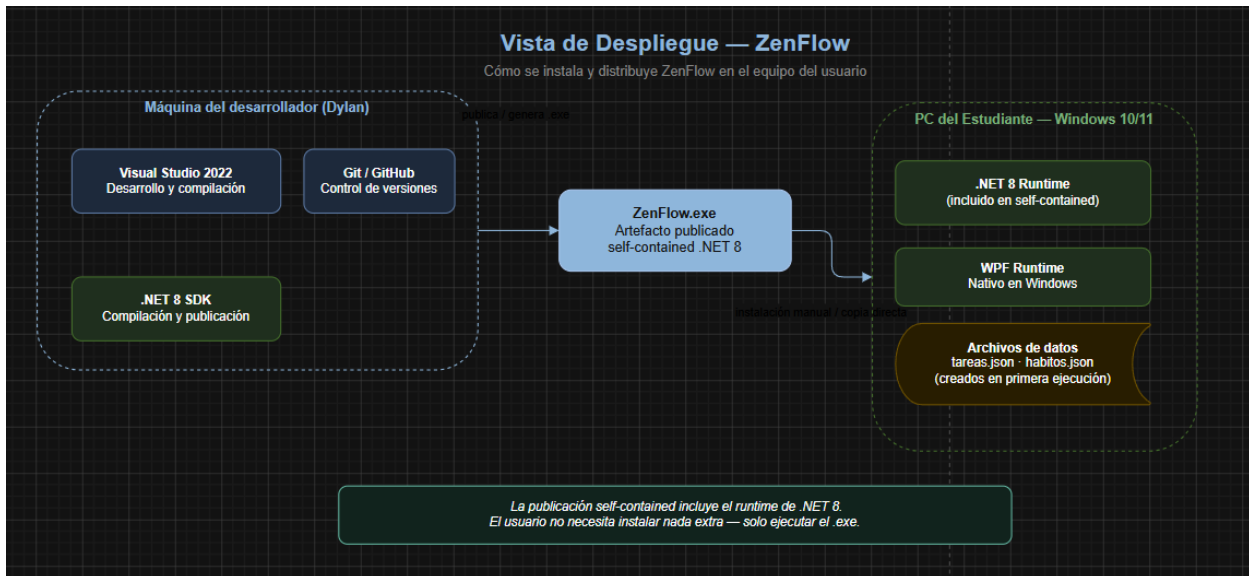
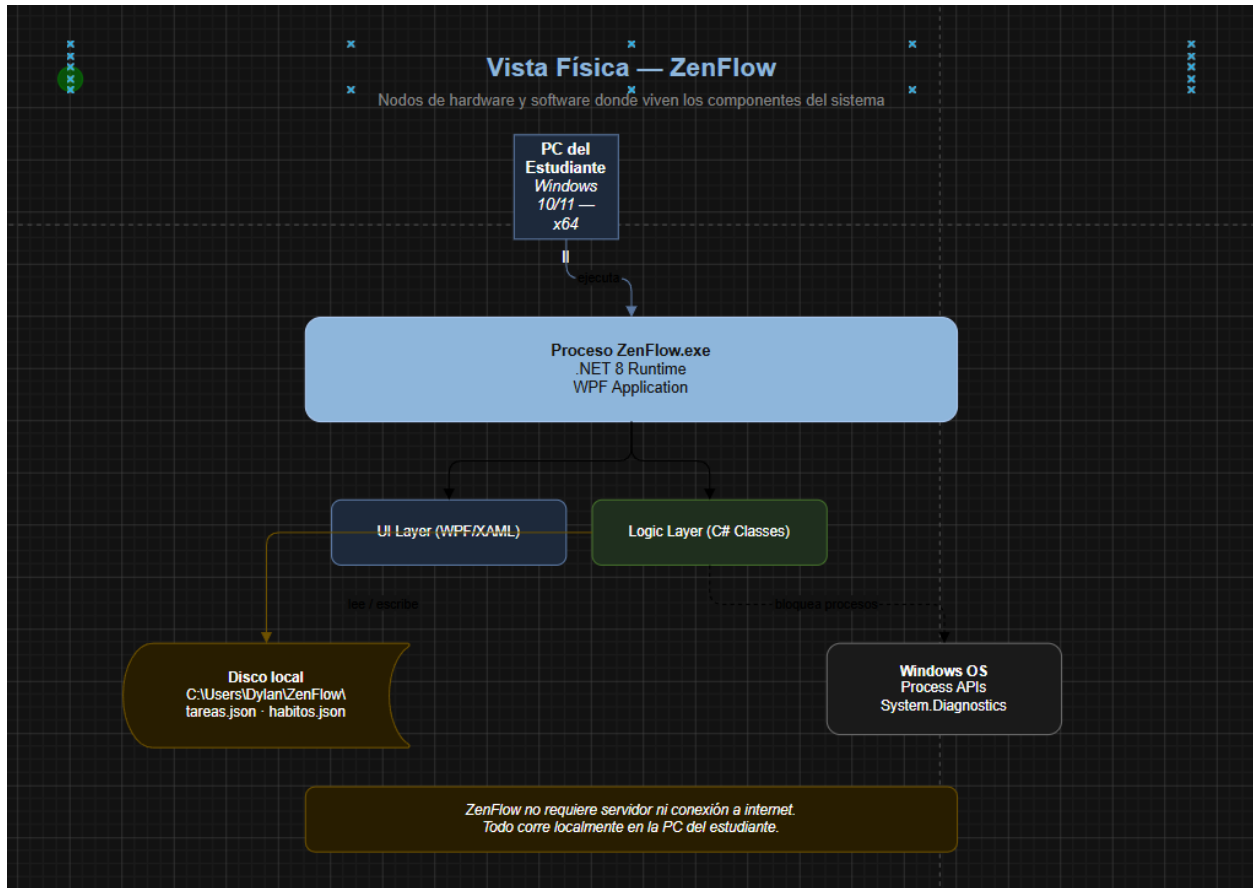
- Técnica: puedo empezar a desarrollar la lógica de negocio de inmediato sin perder tiempo configurando infraestructura. Los archivos JSON son transparentes — puedo abrirlos con cualquier editor y ver los datos directamente, lo que facilita el debugging.
- Técnica: gracias al patrón repositorio, la migración a SQLite en el futuro es quirúrgica — solo afecta TareaRepoJson y HábitoRepoJson. GestorTareas, GestorHábitos, la UI y Program.cs no cambian nada.
- Proceso: al ser los archivos JSON legibles por humanos, puedo verificar que los datos se están guardando correctamente sin herramientas adicionales. Eso acelera el desarrollo y el testing en esta etapa.

Lo que sacrifico o asumo:

- Limitación técnica: JSON no soporta consultas. Si mañana necesito 'todas las tareas vencidas de esta semana', tengo que cargar todo el archivo en memoria y filtrar en código. Con SQLite sería una consulta directa. Por ahora el volumen es pequeño, pero si crece esto se vuelve un problema real.
- Limitación técnica: JSON no tiene soporte para transacciones. Si la app se cierra a la mitad de una escritura, el archivo puede quedar corrupto. SQLite maneja esto automáticamente con su sistema de transacciones ACID.
- Deuda técnica: eventualmente habrá que hacer la migración. Aunque el patrón repositorio la hace sencilla a nivel de código, también hay que migrar los datos existentes del formato JSON al esquema de SQLite. Eso requerirá escribir un script de migración.

Diagrama

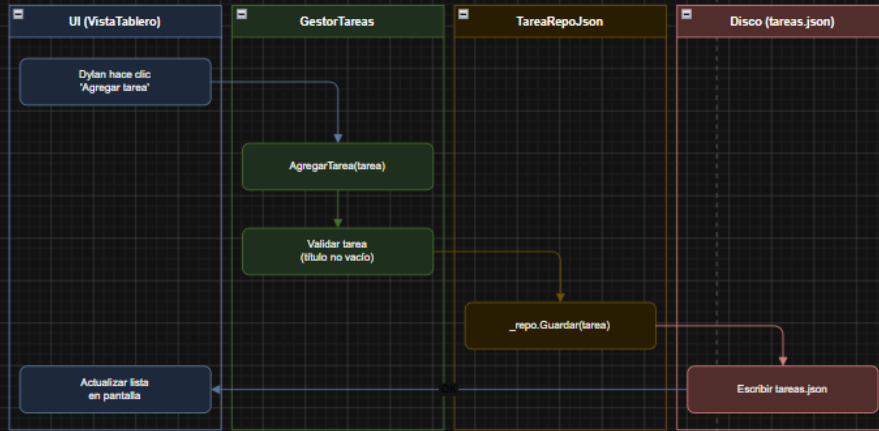




Vista de Procesos — ZenFlow

Flujo de los dos procesos principales: agregar tarea y activar modo enfoque

Flujo 1 — Agregar tarea



Flujo 2 — Activar modo enfoque

